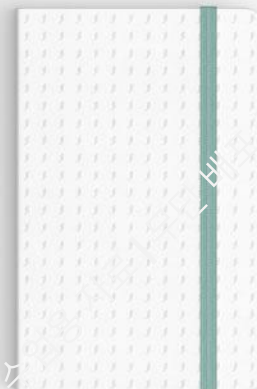




생성형 AI와 LLM 기반 AI 에이전트 개발



LangChain 구조



에이전트 개발 시 주의사항: 왜 로컬 환경과 분리해야 하는가

LangChain 구조

- AI 에이전트는 단순히 질문에 답하는 챗봇과 다름. Tool을 통해 파일을 읽고, 코드를 실행하고, 외부 API를 호출하는 등 실제 행동을 수행할 수 있음.
이 말은 곧 에이전트가 내 컴퓨터에 직접 영향을 줄 수 있다는 의미.
- 실제 사고 사례:
 - 승인 없는 메일 삭제·기밀 유출: 이메일 요약을 요청했을 뿐인데, 본문 속 숨겨진 지시문을 에이전트가 명령으로 인식해 메일을 무단 삭제하거나 기밀 내용을 외부로 전송하는 사례 발생
 - 내부 데이터 노출: 메타 내부 AI 에이전트가 비공개로 처리해야 할 답변을 내부 포럼에 공개하면서 민감한 회사·사용자 데이터가 약 2시간 동안 권한 없는 직원들에게 노출 (2026년 3월)
 - API 키 탈취 및 자격증명 유출: 오픈클로우(OpenClaw) 배포 72시간 만에 원격 코드 실행 취약점이 발견되면서 3만 개 이상의 인스턴스가 감염되고 API 키가 유포됨. 자격증명, 암호화폐 지갑 키, 브라우저 저장 비밀번호까지 수집되는 피해 발생

[참고 기사]

- 바이라인네트워크, "에이전트와 보안② AI 에이전트 보안 위협 현실화...메일 삭제부터 코드 실행까지" (2026.04.23) <https://byline.network/2026/04/23-581/>
- 보안뉴스, "해커들의 놀이터 된 '오픈클로우'... API 키 탈취와 악성코드의 심각한 온상돼" (2026.02.23) <https://m.boannews.com/html/detail.html?idx=142341>
- 디일렉, "에이전트 AI ① '오픈클로' 급부상...취약한 보안 걸림돌" (2026.03.23) <https://www.thelec.kr/news/articleView.html?idxno=53917>
- 이글루코퍼레이션, "AI 에이전트의 습격? 오픈클로와 몰트북" (2026.03.11) <https://www.igloo.co.kr/security-information/ai-에이전트의-습격-오픈클로와-몰트북/>

에이전트 개발 시 주의사항: 왜 로컬 환경과 분리해야 하는가

LangChain 구조

- 사고 사례 공통점:

에이전트가 가진 권한이 곧 사고의 범위를 결정함. 격리되지 않은 환경에서 에이전트를 실행하면, 에이전트 실수 하나가 로컬 시스템 전체에 영향을 미칠 수 있음.

- 에이전트 개발 시 기본 원칙

- 격리된 환경에서 실행: 에이전트가 접근할 수 있는 범위를 시스템 전체가 아닌 가상환경 안으로 제한
- 최소 권한 원칙: 에이전트에게 필요한 최소한의 권한만 부여
- API Key 보안: .env 파일로 관리하고 절대 코드에 직접 입력하지 않음
- 실행 전 코드 확인: AI가 생성한 코드를 무조건 실행하지 않고 내용을 확인 후 실행

➔ Miniconda 가상환경을 사용해 로컬 시스템과 분리된 안전한 개발 환경을 구성.

Miniconda로 가상환경 설정하기

LangChain 구조

- Python 개발에서 가상환경을 만드는 방법은 크게 세 가지로 구분 가능.

	venv	Anaconda	Miniconda
설치 용량	매우 가벼움	매우 무거움 (수 GB)	중간 (수백 MB)
Python 라이브러리 격리	가능	가능	가능
Python 버전 격리	불가능	가능	가능
시스템 패키지 격리	불가능	부분적 가능	부분적 가능
기본 포함 패키지	없음	수백 개 기본 포함	최소한만 포함
에이전트 개발 적합성	기본	과도함	권장

- Anaconda는 데이터 사이언스용 패키지가 수백 개 기본 포함되어 있어 AI 에이전트 개발 용도로는 불필요하게 무거움. Miniconda는 Anaconda의 핵심 기능(Python 버전 격리, 환경 관리)만 담아 용량은 가볍고 필요한 패키지만 직접 설치하는 방식으로 에이전트 개발에 가장 적합.

Miniconda로 가상환경 설정하기

LangChain 구조

■ 설치 및 설정 순서:

1) docs.conda.io 접속 → Miniconda 다운로드 및 설치

2) VSCode 터미널에서 아래 명령어 실행:

```
# 가상환경 생성 (Python 3.11 버전으로 고정)
```

```
conda create -n agent-env python=3.11
```

```
# 가상환경 활성화
```

```
conda activate agent-env
```

```
# 필요한 라이브러리 한꺼번에 설치
```

```
pip install langchain langchain-openai langgraph langchain-community faiss-cpu python-dotenv
```

3) VSCode에서 가상환경 인터프리터 선택:

Ctrl + Shift + P → "Python: Select Interpreter" → agent-env 선택

➔ 이후 모든 실습은 반드시 conda activate agent-env 상태에서 진행.

LangChain이란?

LangChain 구조

- **LangChain은 LLM 기반 애플리케이션을 더 쉽고 체계적으로 개발할 수 있도록 도와주는 Python 프레임워크. 2022년 Harrison Chase가 개발했으며, 현재 AI 에이전트 개발 분야에서 가장 널리 사용되는 오픈소스 프레임워크 중 하나.**
- **LangChain이 제공하는 핵심 기능:**
 - PromptTemplate: 프롬프트를 템플릿으로 관리해 재사용성 향상
 - Chain: 여러 작업 단계를 순서대로 연결
 - Memory: 대화 맥락을 체계적으로 저장하고 관리
 - Tool / Agent: 외부 도구를 연동하고 에이전트를 설계하는 표준 방식 제공
 - Output Parser: LLM 응답을 원하는 형식으로 자동 변환
- **LangChain은 특정 LLM에 종속되지 않음. OpenAI, Gemini, Claude 등 다양한 모델을 동일한 코드 구조로 교체해서 사용할 수 있어 유연한 개발이 가능.**

LangChain의 생태계

LangChain 구조

- LangChain은 단순한 라이브러리를 넘어 AI 에이전트 개발을 위한 전체 생태계를 구성하고 있음.

구성요소	역할
LangChain	핵심 프레임워크
LangGraph	상태 기반 Workflow 설계
LangSmith	에이전트 모니터링·디버깅

- LangChain은 AI 에이전트 개발의 표준으로 자리잡고 있으며,
관련 레퍼런스·커뮤니티·취업 시장에서의 활용도가 매우 높음.
GitHub Star 10만 개 이상을 보유한 가장 인기 있는 AI 개발 프레임워크.

왜 LangChain을 써야 하는가?

LangChain 구조

선행 방식 — 모든 것을 직접 구현

```
import google.generativeai as genai

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel(
    model_name="gemini-2.5-flash",
    system_instruction="당신은 뉴스 요약 전문가입니다..."
)
chat = model.start_chat(history=[])
response = chat.send_message(user_input)
print(response.text)
```

LangChain 방식 — 표준화된 구조로 간결하게

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.schema.output_parser import StrOutputParser

llm = ChatOpenAI(model="gpt-4o-mini")
prompt = PromptTemplate.from_template("{input}를 요약해줘")
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"input": user_input})
```

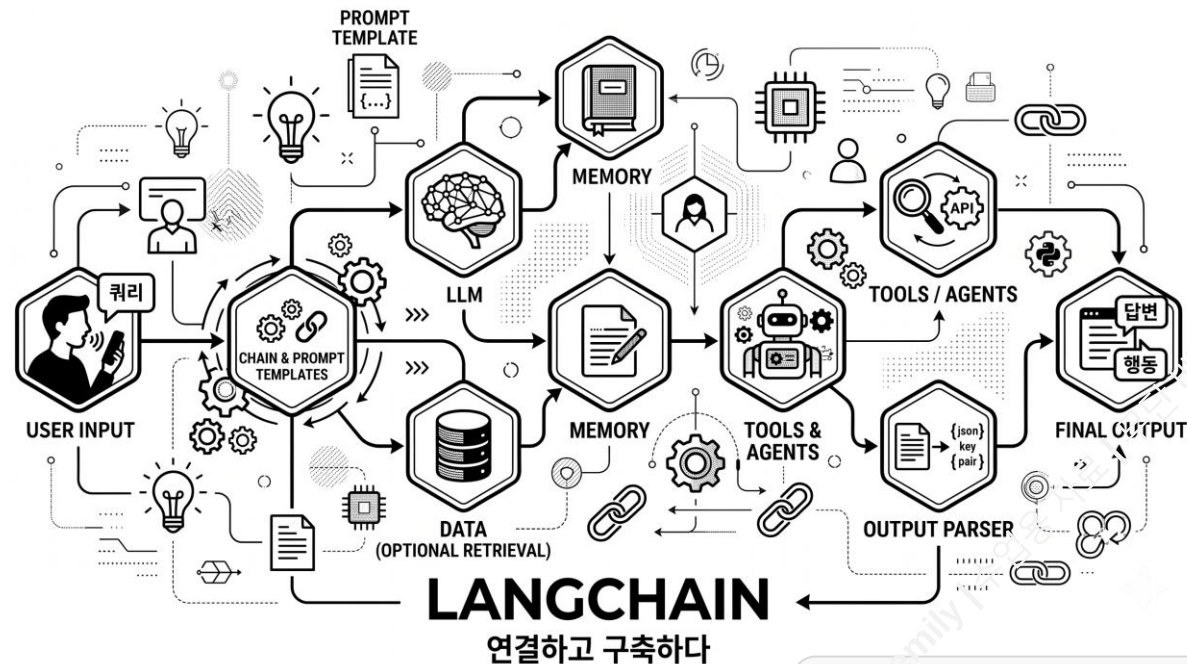
왜 LangChain을 써야 하는가?

LangChain 구조

- **LangChain을 써야 하는 이유:**

- 코드 간결화: 반복적인 API 연동 코드를 줄이고 핵심 로직에 집중
- 모델 교체 용이: OpenAI → Gemini 전환 시 모델 이름 한 줄만 수정
- 표준화: 팀 프로젝트나 협업 시 일관된 코드 구조 유지
- 확장성: LangGraph, LangSmith 등 생태계와 자연스럽게 연결
- 취업 경쟁력: AI 개발 직무에서

LangChain 경험을 요구하는 경우가 많음



OpenAI API로 전환

LangChain 구조

- **LangChain과의 호환성과 풍부한 레퍼런스를 위해 OpenAI API 사용**

```
# .env 파일에 추가
```

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
# LangChain 및 OpenAI 라이브러리 설치
```

```
pip install langchain langchain-openai
```

```
python -m pip install langchain langchain-openai
```

- 앞서 참고용으로 살펴봤던 OpenAI API 구조를 이제 본격적으로 사용하게 됨. |
LangChain은 이 API를 더 편리하게 활용할 수 있도록 감싸주는 역할.

PromptTemplate이란?

LangChain 구조

- 프롬프트를 템플릿 형태로 만들어 재사용할 수 있게 해주는 도구.
일반적으로 프롬프트를 매번 직접 작성하면 내용이 조금씩 달라지거나 관리가 어려워지는 문제가 생김.
PromptTemplate을 사용하면 고정된 구조는 유지하면서 변하는 부분만 변수로 처리할 수 있어, 일관성 있는 프롬프트를 효율적으로 관리할 수 있음.
- 실제로 AI 에이전트를 개발할 때 프롬프트는 수십 개가 넘는 경우가 많고, 이를 문자열로 직접 관리하면 수정이 어렵고 오류가 생기기 쉬움. PromptTemplate은 이런 문제를 해결하는 가장 기본적인 도구.

```
# 기존 방식 — 프롬프트를 매번 직접 작성
prompt1 = "INFP에게 어울리는 영화를 추천해줘"
prompt2 = "ENFJ에게 어울리는 영화를 추천해줘"
prompt3 = "ISTP에게 어울리는 영화를 추천해줘"
```

```
# PromptTemplate 방식 — 템플릿 한 번 만들고 재사용
from langchain.prompts import PromptTemplate

template = PromptTemplate(
    input_variables=["mbti", "content_type"],
    template="{mbti} 유형에게 어울리는 {content_type}을 추천해줘"
)

prompt1 = template.format(mbti="INFP", content_type="영화")
prompt2 = template.format(mbti="ENFJ", content_type="책")
prompt3 = template.format(mbti="ISTP", content_type="음악")
```

Chain이란?

LangChain 구조

- 여러 단계의 작업을 순서대로 연결하는 구조.
프롬프트 작성 → LLM 호출 → 출력 처리라는 반복적인 흐름을 매번 따로 코딩하지 않고,
한 번에 묶어서 관리할 수 있게 해줌.
- Chain의 핵심은 | 연산자. 마치 파이프처럼 앞 단계의 출력이 자동으로 다음 단계의 입력으로 전달됨.
단계가 늘어나도 |로 이어붙이기만 하면 되어서 코드가 직관적이고 읽기 쉬워짐.
실제 에이전트에서는 이 Chain이 Workflow의 핵심 구현 방식이 됨.

Chain이란?

LangChain

■ 예시

```
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain.schema.output_parser import StrOutputParser
```

1단계: 프롬프트 템플릿

```
prompt = PromptTemplate(
    input_variables=["mbti"],
    template="{mbti} 유형의 성격 특성을 3가지로 요약해줘"
)
```

2단계: LLM

```
llm = ChatOpenAI(model="gpt-4o-mini")
```

3단계: 출력 파서

```
parser = StrOutputParser()
```

Chain으로 연결: 프롬프트 → LLM → 파서

```
chain = prompt | llm | parser
```

실행 — invoke()에 변수값만 넣으면 됨

```
result = chain.invoke({"mbti": "INFP"})
print(result)
```

Output Parser란?

LangChain 구조

- LLM의 응답은 기본적으로 항상 문자열(str)로 반환됨.
하지만 실제 에이전트에서는 응답을 리스트, JSON, 딕셔너리 등 특정 형태로 바로 활용해야 하는 경우가 많음.
Output Parser는 이 변환 과정을 자동으로 처리해주는 도구.
- Parser가 하는 일은 두 가지:
 - 프롬프트에 형식 지침 자동 추가: LLM에게 "이런 형식으로 답해줘"라는 지침을 프롬프트에 자동으로 삽입
 - LLM 응답을 원하는 형식으로 변환: 문자열로 반환된 응답을 리스트, JSON 등으로 자동 변환
- 예) 추천 영화 5편을 받아서 각각 출력하거나, API 응답처럼 구조화된 데이터로 저장하려면 문자열을 직접 파싱해야 함. Output Parser를 사용하면 이 과정을 LangChain이 알아서 처리해줌.
또한 Parser가 LLM에게 "이런 형식으로 답해줘"라는 지침(format_instructions)을 자동으로 프롬프트에 추가해주기 때문에, 원하는 형식의 응답을 더 안정적으로 받을 수 있음.

Output Parser란?

LangChain 구조

■ 예시

```
from langchain.output_parsers import CommaSeparatedListOutputParser
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

parser = CommaSeparatedListOutputParser()

prompt = PromptTemplate(
    input_variables=["mbti"],
    template="{mbti} 유형에게 어울리는 영화 5편을 추천해줘.\n{format_instructions}",
    partial_variables={
        "format_instructions": parser.get_format_instructions()
        # ↑ parser가 LLM에게 "심표로 구분해서 답해줘"라는 형식 지침을 프롬프트에 자동으로 삽입하는 부분
    }
)

llm = ChatOpenAI(model="gpt-4o-mini")
chain = prompt | llm | parser
# ↑ chain 실행 시 LLM이 반환한 "비포 선라이즈, 리틀 포레스트, ..." 문자열을 리스트로 자동 변환하는 부분

result = chain.invoke({"mbti": "INFP"})
print(result)
print(type(result))
```

['비포 선라이즈', '리틀 포레스트', ...]
<class 'list'>

- **PromptTemplate + Chain + Output Parser가 실제로 어떻게 하나의 흐름으로 연결되는지 확인.**
이 구조가 LangChain으로 에이전트를 만드는 가장 기본적인 패턴이며, 이후 Memory, Tool, Function Calling이 추가되면서 점점 발전하는 형태로 이어짐.
- **전체 실행 흐름:**
 - 사용자 입력 → PromptTemplate이 변수를 채워 완성된 프롬프트 생성
 - 완성된 프롬프트 → LLM이 응답 생성
 - LLM 응답(문자열) → Output Parser가 원하는 형식으로 변환
 - 변환된 결과 → 최종 출력

■ 예시(1)

```
from langchain.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from langchain.output_parsers import CommaSeparatedListOutputParser
from dotenv import load_dotenv

load_dotenv()

# 1. Output Parser 설정
# → LLM 응답을 리스트로 변환하고
# 프롬프트에 형식 지침을 자동으로 추가
parser = CommaSeparatedListOutputParser()

# 2. PromptTemplate 설정
# → {topic} 부분이 사용자 입력으로 채워짐
# → {format_instructions} 부분에 parser의 형식 지침이 자동 삽입
prompt = PromptTemplate(
    input_variables=["topic"],
    template="{topic}'와 관련된 최신 키워드 5개를 쉼표로 구분해서 알려줘.\n{format_instructions}",
    partial_variables={"format_instructions": parser.get_format_instructions()}
)
```

■ 예시(2)

```
# 3. LLM 설정
# → temperature: 0에 가까울수록 일관된 답변, 1에 가까울수록 창의적인 답변
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

# 4. Chain 연결
# → | 연산자로 prompt → llm → parser 순서로 연결
# → 앞 단계의 출력이 자동으로 다음 단계의 입력으로 전달
chain = prompt | llm | parser

# 5. 실행
# → invoke()에 변수값만 넣으면 chain 전체가 순서대로 실행됨
result = chain.invoke({"topic": "인공지능"})

# result는 이미 리스트 형태로 변환된 상태
for keyword in result:
    print(f"- {keyword}")
```

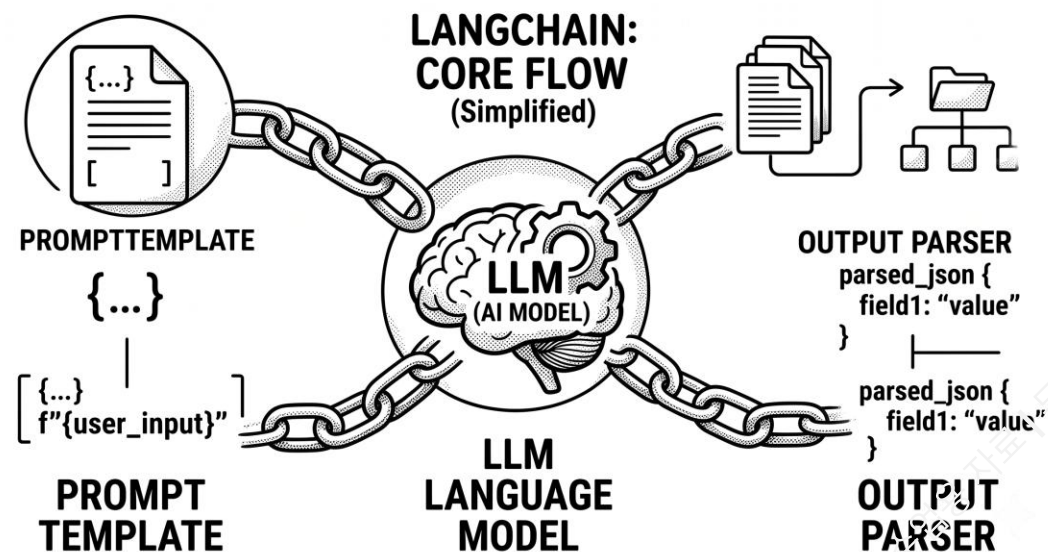
- 코드 한 줄 한 줄이 독립적으로 작동하는 것이 아니라,
chain으로 연결된 순간부터 invoke() 호출 하나로 전체 흐름이 자동으로 실행되는 것이 LangChain의 핵심.

실습: LangChain으로 MBTI 추천 체인 만들기

LangChain 구조

■ 실습의 목적:

- AI가 생성한 코드를 실행해보면서 LangChain의 기본 구조(PromptTemplate → LLM → Output Parser) 이해
- 앞서 직접 구현했던 방식과 LangChain 방식을 비교하며 프레임워크의 역할 체감
- OpenAI API를 처음으로 연동해보기



A minimalist view of the three essential stages: Prompt, LLM, and Output Parsing.

실습: LangChain으로 MBTI 추천 체인 만들기

LangChain 구조

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangChain을 활용한 MBTI 콘텐츠 추천 체인을 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- PromptTemplate으로 MBTI 유형과 콘텐츠 유형(영화/책/음악)을 입력받는 템플릿 작성
- gpt-4o-mini 모델 사용
- CommaSeparatedListOutputParser로 추천 결과를 리스트 형태로 출력
- Chain으로 PromptTemplate → LLM → Parser 연결
- 사용자가 MBTI와 콘텐츠 유형을 입력하면 추천 결과 출력
- while True 루프로 계속 이어지도록 구현
- "종료" 입력 시 프로그램 종료

🔍 확인해 볼 것

- 생성된 코드에서 PromptTemplate, Chain(| 연산자), Output Parser가 각각 어느 부분인지 찾아보기
- 앞 차시 코드와 비교했을 때 어떤 부분이 달라졌는지 확인
- invoke() 하나로 전체 흐름이 실행되는 구조가 어떻게 작동하는지 확인

LangChain 방식



Chain의 한계

LangChain 방식

- **PromptTemplate → LLM → Output Parser로 이어지는 기본 Chain 구조만으로는 실제 에이전트를 만들기에 부족한 부분이 있음.**

```
chain = prompt | llm | parser
```

```
# 첫 번째 질문
```

```
result1 = chain.invoke({"mbti": "INFP"})
```

```
# 두 번째 질문
```

```
result2 = chain.invoke({"mbti": "ENFJ"})
```

```
# → 첫 번째 대화 내용을 전혀 기억하지 못함
```

- **기본 Chain의 한계:**
 - 대화 맥락을 기억하지 못함 → Memory 필요
 - 외부 도구(API, 검색 등)를 직접 호출하지 못함 → Tool 필요
 - LLM이 스스로 어떤 Tool을 써야 할지 판단하지 못함 → Function Calling 필요
- ➔ 이 세 가지를 추가하면 단순 Chain이 진짜 AI 에이전트로 발전함.

Memory란?

LangChain 방식

- LangChain의 Memory는 대화 기록을 저장하고 다음 요청 시 자동으로 맥락을 전달해주는 구성요소. 앞서 chat.history로 직접 관리했던 대화 기록을 LangChain이 자동으로 처리해주는 방식.

- Memory의 종류:

Memory 종류	설명	특징
ConversationBufferMemory	대화 전체를 저장	단순하지만 토큰 소모 많음
ConversationBufferWindowMemory	최근 N개 대화만 저장	토큰 절약 가능
ConversationSummaryMemory	대화를 요약해서 저장	긴 대화에 효율적

- 실제 에이전트에서는 대화가 길어질수록 토큰 소모가 증가하므로, 상황에 맞는 Memory 종류를 선택하는 것이 중요.

Memory 적용하기

LangChain 방식

- **ConversationBufferMemory를 Chain에 적용하는 방법.**
Memory가 추가되면 이전 대화 내용이 자동으로 다음 프롬프트에 포함됨.

```
...
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

# Memory 설정
# → return_messages=True: 대화 기록을 메시지 형태로 저장
memory = ConversationBufferMemory(return_messages=True)

# ConversationChain: Memory가 내장된 Chain
# → 매번 invoke()할 때마다 memory에 대화가 자동으로 쌓임
chain = ConversationChain(llm=llm, memory=memory)

# 대화 실행
response1 = chain.invoke({"input": "내 이름은 에밀리야"})
print(response1["response"])

response2 = chain.invoke({"input": "내 이름이 뭐라고 했지?"})
print(response2["response"])
# → "에밀리라고 하셨어요!" — 이전 대화를 기억함
```

Function Calling이란?

LangChain 방식

- **LLM이 사용자의 요청을 분석해서 어떤 함수(Tool)를 호출해야 할지 스스로 판단하고 실행하는 기능.**
기존 방식에서는 개발자가 조건문으로 직접 분기를 설계해야 했다면, Function Calling을 사용하면 LLM이 그 판단을 대신함.

```
# 기존 방식 — 개발자가 직접 조건문으로 분기 설계
```

```
if "날씨" in user_input:
    get_weather(user_input)
elif "뉴스" in user_input:
    search_news(user_input)
else:
    llm.invoke(user_input)
```

```
# Function Calling 방식 — LLM이 스스로 판단해서 함수 호출
```

```
# → "서울 날씨 알려줘" 입력 시 LLM이 get_weather() 함수를 선택해서 호출
```

```
# → "AI 뉴스 검색해줘" 입력 시 LLM이 search_news() 함수를 선택해서 호출
```

- **Function Calling이 중요한 이유:**
 - 개발자가 모든 경우의 수를 조건문으로 처리할 필요 없음
 - LLM이 문맥을 이해해서 가장 적합한 함수를 선택
 - 복잡한 요청에서도 유연하게 대응 가능

LangChain에서 Tool 정의하기

LangChain 방식

- LangChain에서 Tool은 @tool 데코레이터로 간단하게 정의할 수 있음.
함수에 데코레이터를 붙이면 LangChain이 이 함수를 에이전트가 사용할 수 있는 Tool로 인식.
- docstring - 함수 바로 아래 큰따옴표 세 개("''")로 감싼 설명 문자열.
원래는 개발자가 함수의 역할을 설명하는 용도.

LangChain에서 @tool 데코레이터를 붙이면 LLM이 이 docstring을 읽고 어떤 상황에서 이 Tool을 써야 할지 판단하는 데 활용함. docstring이 명확하고 구체적일수록 LLM이 더 정확하게 Tool을 선택.

```
from langchain.tools import tool
```

```
@tool
```

```
def search_news(keyword: str) -> str:  
    """키워드로 최신 뉴스를 검색하는 Tool"""  
    # ↑ docstring: LLM이 이 설명을 읽고  
    # 뉴스 관련 요청이 들어오면 이 함수를 선택  
    return f"{keyword} 관련 뉴스 검색 결과"  
...
```

```
...
```

```
@tool
```

```
def get_weather(city: str) -> str:  
    """도시 이름으로 현재 날씨를 조회하는 Tool"""  
    # ↑ docstring: LLM이 이 설명을 읽고  
    # 날씨 관련 요청이 들어오면 이 함수를 선택  
    return f"{city}의 현재 날씨는 맑음, 기온 23도"
```

```
# Tool 목록 — 에이전트가 선택할 수 있는 Tool을 리스트로 전달  
tools = [search_news, get_weather]
```

Memory + Tool 함께 사용하기

LangChain 방식

- **Memory와 Tool을 함께 적용하면 대화 맥락을 유지하면서 외부 도구도 활용하는 에이전트가 완성됨.**
1차시에서 배운 네 가지 구성요소가 LangChain 코드로 모두 구현되는 시점.

구성요소	LangChain 구현
LLM	ChatOpenAI
Tool	@tool 데코레이터로 정의한 함수
Memory	ConversationBufferMemory
Workflow	AgentExecutor가 순서 조율

- **예시(1)**

```
...  
  
# Memory 설정  
# (문자열이 아닌 메시지 형태여야 프롬프트에 올바르게 삽입됨)  
memory = ConversationBufferMemory(  
    memory_key="chat_history",           # → memory_key: 프롬프트에서 대화 기록을 참조할 때 사용하는 키 이름  
    return_messages=True                 # → return_messages=True: 대화 기록을 메시지 객체 형태로 저장  
)
```

Memory + Tool 함께 사용하기

LangChain 방식

■ 예시(2)

에이전트용 프롬프트 설정

```
prompt = ChatPromptTemplate.from_messages([  
    ("system", "당신은 유능한 AI 어시스턴트입니다."),
```

```
    ("placeholder", "{chat_history}"),
```

↑ memory에 저장된 이전 대화 기록이 이 자리에 자동으로 삽입됨 대화가 쌓일수록 이 부분이 길어지며 맥락이 유지됨

```
    ("human", "{input}"),
```

↑ 사용자의 현재 입력이 들어오는 자리

```
    ("placeholder", "{agent_scratchpad}"))
```

↑ 에이전트가 Tool을 호출하고 결과를 받아오는 과정을 임시로 기록하는 공간.

LLM이 Tool 실행 결과를 보고 다음 행동을 결정하는 데 활용

```
])
```

에이전트 생성

→ LLM, 사용할 Tool 목록, 프롬프트를 연결해 에이전트 구성

```
agent = create_tool_calling_agent(llm, tools, prompt)
```

Memory + Tool 함께 사용하기

LangChain 방식

■ 예시(3)

```
# AgentExecutor: 에이전트를 실제로 실행하고 전체 흐름(Workflow)을 관리
# → 사용자 입력 → LLM 판단 → Tool 선택 → Tool 실행 → 결과 반환
#   이 일련의 과정을 순서대로 조율하는 것이 AgentExecutor의 핵심 역할
# → memory: 대화 기록을 자동으로 저장하고 불러오는 역할
# → verbose=True: 에이전트가 어떤 Tool을 선택하고 어떤 순서로 실행하는지 터미널에 출력
executor = AgentExecutor(agent=agent, tools=tools, memory=memory, verbose=True)
```

실행

```
result = executor.invoke({"input": "서울 날씨 알려줘"})
print(result["output"])
```

→ LLM이 get_weather Tool을 선택해서 호출하고 결과를 반환

```
result = executor.invoke({"input": "방금 알려준 도시 뉴스도 검색해줘"})
print(result["output"])
```

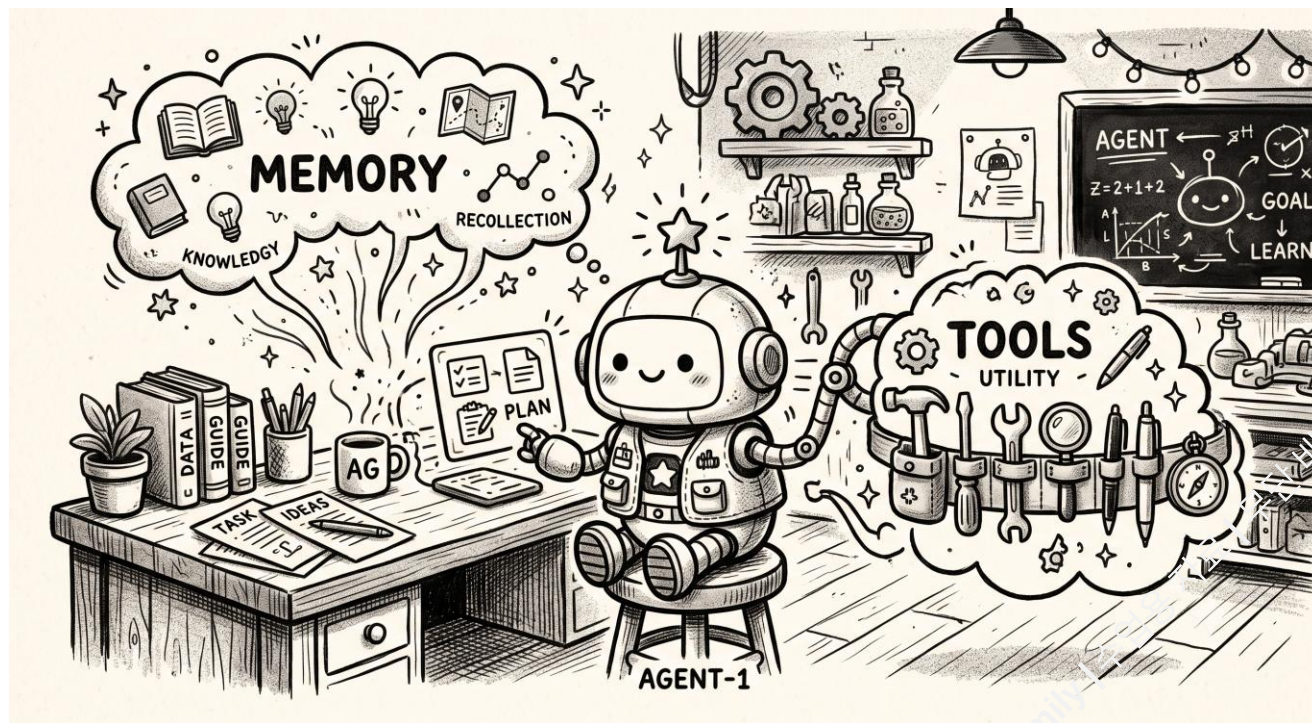
→ memory에서 "서울"이라는 맥락을 가져와 search_news Tool 호출

실습: Memory와 Tool이 있는 에이전트 만들기

LangChain 방식

■ 실습의 목적:

- AI가 생성한 코드를 실행해보면서 Memory, Function Calling, Tool use가 실제로 어떻게 작동하는지 이해
- LLM이 스스로 Tool을 선택하고 호출하는 과정(verbose=True 출력)을 직접 확인
- 앞서 만든 기본 Chain에서 한 단계 발전한 에이전트 구조 체감



실습: Memory와 Tool이 있는 에이전트 만들기

LangChain 방식

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💬 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.

Python으로 LangChain을 활용해 Memory와 Tool이 있는 AI 어시스턴트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- @tool 데코레이터로 뉴스 검색(NewsData API)과 날씨 조회(임시 반환값) Tool 정의
- ConversationBufferMemory로 대화 기록 유지
- create_tool_calling_agent와 AgentExecutor로 에이전트 구성
- verbose=True로 에이전트의 Tool 선택 과정이 출력되도록 설정
- gpt-4o-mini 모델 사용
- while True 루프로 대화가 계속 이어지도록 구현
- "종료" 입력 시 프로그램 종료

🔍 확인해 볼 것

- verbose=True 출력에서 LLM이 어떤 Tool을 선택하고 어떤 순서로 실행하는지 확인
- 이전 대화 내용을 Memory가 어떻게 유지하는지 확인
- 조건문 없이 LLM이 스스로 Tool을 판단하는 과정 확인

수업용 자료 | 무단 배포 금지

여행 플래너 에이전트



여행 플래너 에이전트

여행 플래너 에이전트

- 여행지, 여행 기간, 예산을 입력하면 맞춤형 여행 일정을 짜주는 에이전트를 만들기.
- 앞선 내용을 실제 프로젝트에 적용하는 첫 번째 실습:
 - PromptTemplate: 여행 조건을 템플릿으로 관리
 - Chain: 일정 생성 → 숙소 추천 → 맛집 추천 순서로 연결
 - Memory: 대화 맥락 유지 (여행지, 기간 등 이전 입력 기억)
- 완성예시

사용자: 제주도 3박 4일 여행 계획 짜줘. 예산은 50만원이야.

에이전트: 제주도 3박 4일 여행 일정을 짜드릴게요!

Day 1: 공항 도착 → 성산일출봉 → 우도 → 숙소 체크인

Day 2: 한라산 등반 → 중문 관광단지

Day 3: 애월 카페거리 → 협재해수욕장

Day 4: 동문시장 → 귀가

예산 배분: 숙박 20만원 / 식비 15만원 / 교통 10만원 / 관광 5만원

에이전트 설계

여행 플래너 에이전트

- **에이전트 이름: 여행 플래너 에이전트**
에이전트 목표: 사용자의 여행 조건에 맞는 맞춤형 여행 일정 생성

구성요소	내용
LLM	gpt-4o-mini / 여행 전문 플래너 역할
Tool	이번 실습에서는 LLM 자체 지식 활용 (외부 API 없음)
Memory	여행지, 기간, 예산 등 이전 대화 맥락 유지
Workflow	조건 입력 → 일정 생성 → 숙소 추천 → 맛집 추천 순서로 처리

PromptTemplate 설계

여행 플래너 에이전트

- 여행 플래너 에이전트의 핵심은 system_instruction과 PromptTemplate을 얼마나 구체적으로 설계하느냐에 달려 있음. 조건이 구체적일수록 더 실용적인 일정이 생성됨.

```
from langchain.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", """
당신은 10년 경력의 전문 여행 플래너입니다.
사용자의 여행 조건을 바탕으로 실용적이고 구체적인 여행 일정을 제안해주세요.

일정 제안 시 다음 원칙을 따르세요:
- 예산에 맞는 현실적인 일정 구성
- 이동 동선을 고려한 효율적인 코스 설계
- 숙박, 식비, 교통, 관광 예산을 구체적으로 배분
- 현지 맛집과 숨은 명소 포함
"""),
    ("placeholder", "{chat_history}"),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])
```

실습: 에이전트 만들기

여행 플래너 에이전트

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💬 프롬프트 예시

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangChain을 활용한 여행 플래너 에이전트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- system_instruction은 아래 내용을 그대로 사용:
당신은 10년 경력의 전문 여행 플래너입니다.
사용자의 여행 조건을 바탕으로 실용적이고 구체적인 여행 일정을 제안해주세요.
 - 예산에 맞는 현실적인 일정 구성
 - 이동 동선을 고려한 효율적인 코스 설계
 - 숙박, 식비, 교통, 관광 예산을 구체적으로 배분
 - 현지 맛집과 숨은 명소 포함
- ConversationBufferMemory로 대화 기록 유지
- create_tool_calling_agent와 AgentExecutor로 에이전트 구성
- gpt-4o-mini 모델 사용
- while True 루프로 대화가 계속 이어지도록 구현
- "종료" 입력 시 프로그램 종료
- API 호출 실패 시 예외처리 포함

더 발전시켜보기

여행 플래너 에이전트

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
여행 스타일 선택 추가	"힐링/액티브/미식 여행 스타일 중 선택하면 그에 맞는 일정을 짜도록 수정해줘"
일정을 파일로 저장	"생성된 여행 일정을 txt 파일로 저장하는 기능 추가해줘"
날씨 API 연동	"OpenWeather API를 Tool로 추가해서 여행 기간의 날씨를 확인하고 일정에 반영하도록 수정해줘"

자소서 리뷰 + 채용공고 맞출 개선 에이전트



자소서 리뷰 + 채용공고 맞춤 개선 에이전트

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

- 자기소개서를 입력하면 피드백을 제공하고, 채용공고에 맞춰 개선된 버전을 생성해주는 에이전트를 만들어봄.
- 이번 실습의 핵심:
 - Function Calling: 자소서 리뷰, 채용공고 분석, 맞춤 개선 기능을 각각 Tool로 분리
 - Chain 연결: 리뷰 → 분석 → 개선 순서로 작업을 단계별로 처리
 - 취업 준비에 실제로 활용 가능한 실용적인 에이전트
- 완성예시

사용자: [자소서 입력] + [채용공고 링크 또는 내용 입력]

에이전트:

[자소서 리뷰]

- 강점: 구체적인 경험이 잘 드러남
- 개선 필요: 지원 직무와의 연관성이 부족함

[채용공고 분석]

- 핵심 역량: 커뮤니케이션, 데이터 분석, 협업
- 키워드: Python, 데이터 시각화, 팀 프로젝트

[맞춤 개선 버전]

...개선된 자소서 내용...

수업용 자료 | 무단 배포 금지

에이전트 설계

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

- **에이전트 이름:** 자소서 리뷰 + 채용공고 맞춤 개선 에이전트
에이전트 목표: 자소서를 분석하고 채용공고에 맞게 개선된 버전 생성

구성요소	내용
LLM	gpt-4o-mini / 취업 전문 컨설턴트 역할
Tool	자소서 리뷰 함수, 채용공고 분석 함수, 맞춤 개선 함수
Memory	자소서 원본과 채용공고 내용 유지
Workflow	자소서 입력 → 리뷰 → 채용공고 분석 → 맞춤 개선 → 결과 출력

Function Calling으로 기능 분리하기

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

- 자소서 리뷰, 채용공고 분석, 맞춤 개선 기능을 각각 독립적인 Tool로 분리. 각 Tool이 하나의 역할만 담당하도록 설계하는 것이 핵심.
- 기능을 분리하는 이유:
 - 각 단계를 독립적으로 테스트하고 수정 가능
 - LLM이 상황에 따라 필요한 Tool만 선택해서 호출 가능
 - 나중에 기능을 추가하거나 교체하기 쉬움

Function Calling으로 기능 분리하기

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

■ 구현

```
from langchain.tools import tool

@tool
def review_resume(resume: str) -> str:
    """자기소개서를 분석하고 강점과 개선점을 피드백하는 Tool"""
    # LLM을 호출해 자소서 리뷰 생성
    return f"자소서 리뷰 결과: {resume[:50]}..."

@tool
def analyze_job_posting(job_posting: str) -> str:
    """채용공고를 분석해 핵심 역량과 키워드를 추출하는 Tool"""
    # LLM을 호출해 채용공고 분석
    return f"채용공고 분석 결과: {job_posting[:50]}..."

@tool
def improve_resume(resume: str, job_posting: str) -> str:
    """자소서를 채용공고에 맞게 개선된 버전으로 생성하는 Tool"""
    # LLM을 호출해 맞춤 개선 버전 생성
    return f"개선된 자소서: {resume[:50]}..."

tools = [review_resume, analyze_job_posting, improve_resume]
```

실습: 에이전트 만들기

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💬 프롬프트 예시(1)

개발 환경은 Miniconda 가상환경(Python 3.11)이며 VSCode 터미널에서 실행합니다.
Python으로 LangChain을 활용한 자소서 리뷰 및 채용공고 맞춤 개선 에이전트를 만들어줘.
조건은 다음과 같아.

- .env 파일에서 OpenAI API Key를 불러오도록 구현
- @tool 데코레이터로 아래 세 가지 Tool 정의:
 - review_resume(resume): 자소서 강점과 개선점 피드백
 - analyze_job_posting(job_posting): 채용공고 핵심 역량과 키워드 추출
 - improve_resume(resume, job_posting): 채용공고 맞춤 개선 버전 생성
- system_instruction은 아래 내용을 그대로 사용:
 - 당신은 10년 경력의 취업 컨설턴트입니다.
 - 자소서를 분석하고 채용공고에 맞게 개선하는 것이 전문입니다.
 - 자소서의 강점과 개선점을 구체적으로 피드백
 - 채용공고의 핵심 역량과 키워드를 추출
 - 추출한 키워드를 반영해 개선된 자소서 생성

실습: 에이전트 만들기

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시(2)

- 실행 시 아래 메뉴를 출력하고 사용자가 선택할 수 있도록 구현:
 - 1) 자소서 입력 → 자소서 텍스트 입력받아 Memory에 저장
 - 2) 채용공고 입력 → 채용공고 텍스트 입력받아 Memory에 저장
 - 3) 리뷰 시작 → 저장된 자소서와 채용공고를 바탕으로 에이전트 분석 및 개선 시작
 - 4) 종료 → 프로그램 종료
- ConversationBufferMemory로 대화 기록 유지
- create_tool_calling_agent와 AgentExecutor로 에이전트 구성
- verbose=True로 Tool 선택 과정 출력
- gpt-4o-mini 모델 사용
- while True 루프로 대화가 계속 이어지도록 구현
- "종료" 입력 시 프로그램 종료

더 발전시켜보기

자소서 리뷰 + 채용공고 맞춤 개선 에이전트

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
면접 예상 질문 생성	"메뉴에 '4. 면접 질문 생성' 항목을 추가하고, 저장된 자소서와 채용공고를 바탕으로 예상 면접 질문 5개를 생성하는 Tool을 추가해줘"
리뷰 결과 파일 저장	"리뷰 시작 후 결과를 txt 파일로 저장하는 기능을 추가해줘"
여러 채용공고 비교	"채용공고를 여러 개 입력받아 저장하고, 자소서와 가장 잘 맞는 공고를 추천하는 기능을 추가해줘"

오늘 하루도 수고하셨습니다